Instead, there are four keys at distance 2: “a”, “an”, “and”, “anti”; they can all be returned.

After computing our vector of distances, we traverse any outgoing edge and repeat the process for the next nodes. In this case, there is only one, associated with the string “a”, the concatenation of the labels leading to it; we can build the next row in the table using the previous row only, and comparing the last character (the one marking the last edge traversed) to each character in s (note that for the empty string column, the distance will always be the length of the path).

Therefore, the second row in our table will start with a 1, then have 0 in cell [1,1], because both strings start with an ‘a’; for the next character in s, there isn’t a corresponding char in the node’s key (because it’s shorter), so we need to add 1 to have its distance, and same for the last character. In fact, as a double check, if we consider the prefix “am”, the distance to “a” is 1, while for “amt” is 2.

Notice that the cost to compare two strings is always contained in the bottom-right cell, so for “a” and “amt” this distance is 2.

The algorithm goes on traversing **all** branches, until we get to a point where the cost can’t decrease any more (when the path is already as long as the string s or longer, there is no point in going down a branch, as soon as we find a key node) or all the distances in the last row are larger than a user-defined threshold, the max meaningful distance: this is particularly useful when searching long strings, that would otherwise cause most of the tree to be traversed (while,